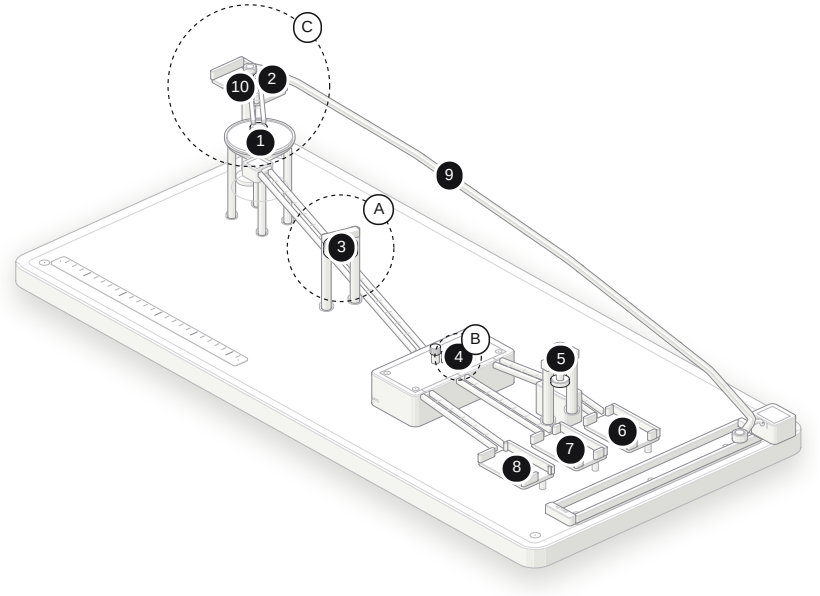
Minted: 0



What does one request cost the garbage collector?

Every exchange runs one coin round the machine. The reading bridge is Glyph11: headers become zero-copy ByteString views over the request buffer — no strings minted. The gate is the handler tree, the press is serialization, and the return chute empties back into the pool.

0 minted
ALLOCATIONS ON THE WHOLE EXCHANGE

1024 deep
PER-THREAD STACK OF POOLED REQUESTS

0 copies
HEADERS ARE VIEWS OVER THE READ BUFFER

20 bytes
OF DIGIT BUFFER COVER ANY CONTENT-LENGTH

PARTS

01 **Hopper**
Accepted connections queue here to be dealt one at a time.

02 **The coin**
One pooled Request, reused every exchange.

03 **Reading bridge**
Glyph11 weighs the headers into zero-copy views.

04 **Routing gate**
The handler tree's answer: response, null, or thrown.

05 **The press**
Status line and headers stamped out as raw UTF-8.

06 **Handler tray**
Where answered requests land before the return.

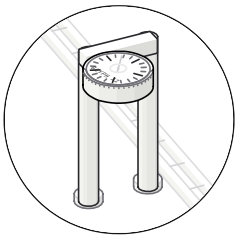
07 **404 tray**
A null answer drops the coin here.

08 **Error tray**
A thrown one is mapped to a status and lands here.

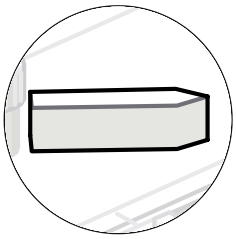
09 **Return tube**
Drained, reset and blown back to the pool.

10 **The pool**
Spare coins: 1024 Requests deep, per thread.

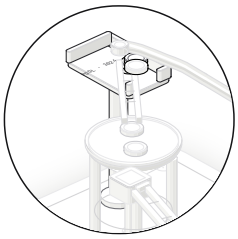
CLOSE-UPS



(A) **The gauge**
The dial swings as the bridge reads: parsing cost, weighed in allocations, pinned at zero.

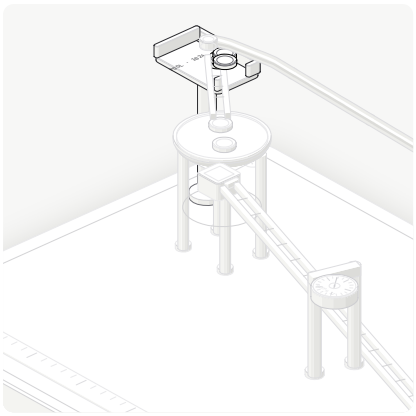


(B) **The gate**
Three lanes, labelled handler, 404 and error — the three shapes an IHandler answer can take.



(C) **The pool**
Coins stacked and waiting. Rent, run, return: the pool makes allocation a decision, not an accident.

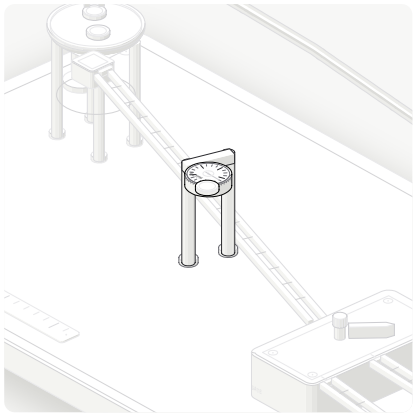
How it works



01

One coin, dealt again

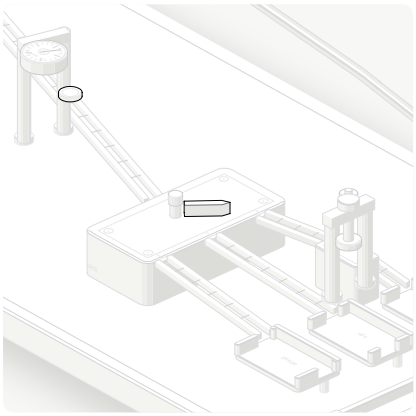
The request object is rented from a per-thread stack — up to 1024 deep. The same coin runs the machine forever; the readout never minted one.



02

The reading bridge

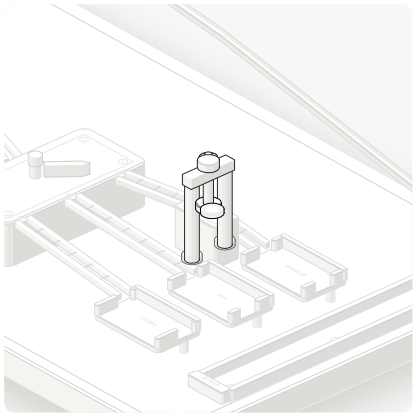
Glyph11's hardened parser reads the head of the stream: every header becomes a ByteString view over bytes already in memory, hash precomputed.



03

The gate decides

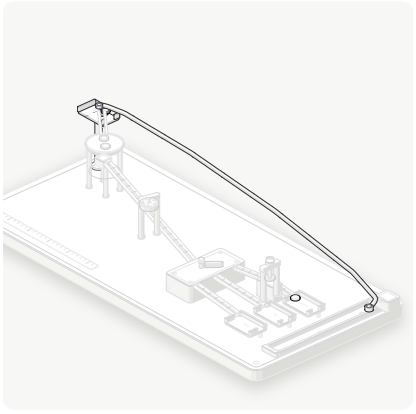
The handler tree answers. A response routes on; a null drops the coin in the 404 tray; a thrown error is mapped and takes the third lane.



04

The stamping press

Serialization: status line and headers as raw UTF-8 — server and date cached — then the body drives a Regular or Chunked sink.



05

Back to the tray

Drain unread bodies, reset the state, re-pool the object. If bytes are already buffered, the next request parses immediately.

WHY VIEWS, NOT STRINGS?

A header read as a C# string allocates on every request and again for every lookup hash. A ByteString is a struct over the bytes already in the buffer, with its hash computed once by the [MemoryView] generator — parsing, matching and hashing stop costing anything at all.

SPECIFICATION

Parser	Glyph11, UltraHardenedParser	Bodies	Pipelines — streamed, not buffered
Headers	ByteString views, hash precomputed	Sinks	RegularSink or ChunkedSink
Generated	[MemoryView] source generator	Pipelining	next request parses from the buffer
Requests	[ThreadStatic] pool, 1024 max		